

Formal Criteria-Based Test Design & Automation

Instructor: Dr. Bashar Al Shboul

Prepared by:

Sara fraihat 2332395
Sara Shammout 2239048
Noor Hammed 2335905

1. Model-Driven Test Design (MDTD): Input Space Partitioning Characteristics and Logical Blocks

We partitioned the input space of the DiscountCalculator based on three main characteristics to ensure disjoint and complete coverage

Characteristic	Block 1	Block 2	Block 3
C1: customerType	"NEW"	"REGULAR"	"PREMIUM"
C2: totalOrdersInLastYear	$x < 10$	$x \geq 10$	N/A
C3: isSubscribedToNewsletter	true	false	N/A

2. Pair-Wise Coverage (PWC) Test Cases

To achieve **Pair-Wise Coverage**, we derived a set of combinations where every possible pair of values between any two characteristics is covered at least once

Test Case	customerType	totalOrders	isSubscribed	Expected Output / Result
TC1	"NEW"	5 (Block 1)	true	7.0% Discount (Base 5 + News 2 + New 0)
TC2	"REGULAR"	12 (Block 2)	false	13.0% Discount (Base 5 + Reg 3 + Loyalty 5)
TC3	"PREMIUM"	8 (Block 1)	false	10.0% Discount (Base 5 + Prem 5)
TC4	"PREMIUM"	15 (Block 2)	true	15.0% Discount (Capped at 15%)
TC5	"REGULAR"	4 (Block 1)	true	10.0% Discount (Base 5 + Reg 3 + News 2)
TC6	"NEW"	10 (Block 2)	false	Infeasible (IllegalArgumentException)

3. Infeasible Test Case Justification

Justification for TC6: The combination in TC6 is logically **Infeasible**. By business logic, a customer marked as "NEW" cannot have a history of **10 or more orders** from the previous year. This scenario is handled by the system using input validation, which throws an `IllegalArgumentException` to maintain data integrity.

4. Limitations of Pair-Wise Testing

While Pair-Wise Coverage (PWC) is highly efficient in reducing the number of test cases while maintaining a high fault-detection rate, it has inherent limitations. The primary weakness is that it only guarantees testing of interactions between **any two** parameters. It does not account for **3-way or higher-order interactions**, which means complex logic errors triggered by a specific combination of three or more variables may go undetected. In our system, if a bug only occurs when a specific customerType, orderCount, and subscriptionStatus are all combined in a certain way, PWC might not catch it unless that specific triplet was accidentally included in our pairs

5. Team Contribution Matrix

Student Name	Role	Primary Responsibilities & Deliverables
Sarah Freihat	Logic Designer	Designed ISP blocks and characteristics. Derived the Pair-Wise test cases and implemented the DiscountCalculator.java logic.
Sarah Shammout	Automation Lead	Developed the DiscountCalculatorTest.java class. Implemented automated @ParameterizedTest and exception handling using JUnit
Noor Hammed	QA	Conducted the limitations analysis of Pair-Wise testing. Managed the final documentation, report formatting (PDF), and version control synchronization on GitHub.